

Laporan Tugas Besar 2

IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model

Semester II Tahun 2025/2026



Nama Kelompok: YangPentingAda

Aziza Dharma Putri 13524017

Keisha Rizka Syofyani 13524073

Vara Azzara Ramli Pulukadang 13524091

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena atas rahmat-Nya, laporan Tugas Besar 2 IF2211 Strategi Algoritma ini dapat diselesaikan. Laporan ini membahas perancangan dan implementasi aplikasi web untuk melakukan penelusuran CSS selector pada pohon Document Object Model (DOM) menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS).

Kami mengucapkan terima kasih kepada dosen pengampu, asisten mata kuliah, serta seluruh pihak yang telah membantu proses pengerjaan tugas besar ini. Kami menyadari bahwa laporan dan aplikasi ini masih dapat dikembangkan lebih lanjut. Oleh karena itu, kritik dan saran sangat kami harapkan untuk perbaikan ke depannya.

Bandung, 25 April 2026

Kelompok YangPentingAda

DAFTAR ISI

KATA PENGANTAR

BAB 1	DESKRIPSI TUGAS	1
BAB 2	LANDASAN TEORI	2
2.1	Breadth First Search	2
2.2	Depth First Search	2
2.3	Document Object Model	3
2.4	Tag HTML	4
2.5	CSS Selector	4
2.6	HTML Parsing	5
2.7	Lowest Common Ancestor dan Binary Lifting	6
BAB 3	APLIKASI BFS DAN DFS	7
3.1	Aplikasi BFS	7
3.1.1	Konsep BFS pada DOM Tree	7
3.1.2	Struktur Data Queue	7
3.1.3	Alur Kerja Traversal BFS	8
3.1.4	Contoh Urutan Traversal BFS	8
3.1.5	Kecocokan BFS untuk Berbagai Skenario	9
3.2	Aplikasi DFS	9
3.2.1	Konsep DFS pada DOM Tree	9
3.2.2	Struktur Data Stack	9
3.2.3	Reverse Push untuk Mempertahankan Urutan Natural HTML	10
3.2.4	Pre-order Iteratif	10
3.2.5	Alur Kerja Traversal DFS	11
3.2.6	Contoh Urutan Traversal DFS	11
3.3	Perbandingan BFS dengan DFS	11
3.3.1	Cara Kerja Traversal	11
3.3.2	Kompleksitas Waktu	12
3.3.3	Kompleksitas Memori	12
3.3.4	Pengaruh Karakteristik DOM Tree	12

3.3.5	Pengaruh Limit Hasil Pencarian	13
3.3.6	Kecocokan Algoritma untuk Berbagai Skenario Selector	13
3.3.7	Tabel Perbandingan	14
BAB 4	IMPLEMENTASI DAN PENGUJIAN	15
4.1	Implementasi Program Secara Umum	15
4.2	Struktur Direktori dan Modul Program	15
4.3	Implementasi Backend	17
4.4	Implementasi Scraper dan Parser	17
4.5	Implementasi BFS dan DFS	17
4.6	Implementasi CSS Selector	18
4.7	Implementasi Frontend	18
4.8	Implementasi Visualisasi dan Traversal Log	18
4.9	Implementasi Docker dan Deployment	19
4.10	Pengujian Program	19
BAB 5	KESIMPULAN DAN SARAN	26
5.1	Kesimpulan	26
5.2	Saran	26
LAMPIRAN		27
PERNYATAAN		30

BAB 1

DESKRIPSI TUGAS

Pada tugas ini, kami diminta untuk membuat aplikasi traversal pohon HTML atau Document Object Model (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Aplikasi melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM dari sebuah website atau teks HTML yang diberikan pengguna.

Aplikasi yang dibuat berbasis web. Frontend dibangun menggunakan JavaScript dengan framework React, sedangkan backend dibangun menggunakan TypeScript. Backend bertugas mengambil HTML dari URL atau input manual, melakukan parsing HTML menjadi DOM Tree, menjalankan algoritma pencarian, dan mengirimkan hasil pencarian ke frontend.

Input yang difasilitasi oleh aplikasi adalah sebagai berikut:

1. URL website atau teks HTML manual.
2. Pilihan algoritma traversal, yaitu BFS atau DFS.
3. CSS selector.
4. Jumlah hasil yang ingin ditampilkan, yaitu seluruh kemunculan atau *top-n* kemunculan.

HTML yang diperoleh kemudian diparsing menjadi struktur data pohon. Setelah itu, algoritma pencarian yang dipilih akan menelusuri DOM Tree dan menentukan elemen yang sesuai dengan selector. Output yang ditampilkan oleh aplikasi meliputi:

1. Visualisasi struktur DOM Tree beserta kedalaman maksimum tree.
2. Highlight pada node yang dikunjungi dan node yang cocok dengan selector.
3. Waktu pencarian.
4. Jumlah node yang dikunjungi.
5. Traversal log yang berisi tahapan penelusuran.

Selain spesifikasi wajib, tugas ini juga menyediakan beberapa komponen bonus. Implementasi bonus yang dikerjakan dijelaskan pada bagian implementasi dan pengujian.

BAB 2

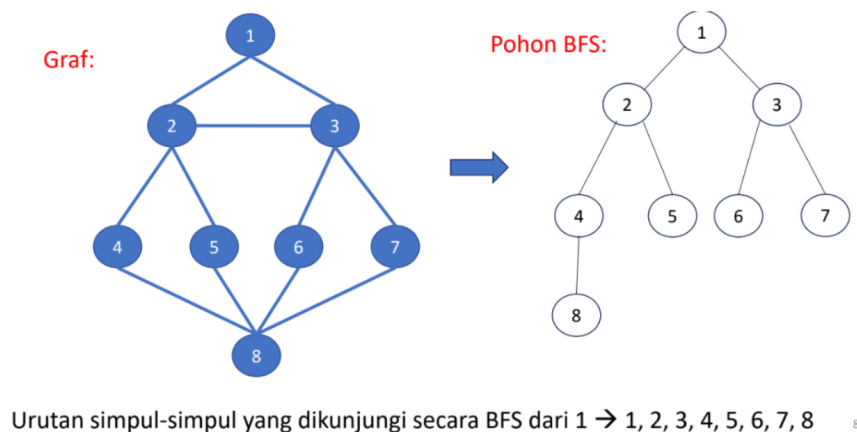
LANDASAN TEORI

2.1 Breadth First Search

Breadth First Search (BFS) adalah algoritma traversal graf atau pohon yang mengunjungi node secara melebar. Pada struktur pohon, BFS memproses node berdasarkan kedalaman. Node pada kedalaman yang lebih kecil akan diproses terlebih dahulu sebelum node pada kedalaman yang lebih besar.

BFS menggunakan struktur data queue dengan prinsip FIFO (*First In, First Out*). Node root dimasukkan ke dalam queue terlebih dahulu. Selama queue tidak kosong, node paling depan dikeluarkan, diproses, lalu seluruh anaknya dimasukkan ke bagian belakang queue. Proses ini berlanjut hingga node yang dicari ditemukan atau semua node selesai diproses.

Pada DOM Tree, BFS cocok digunakan ketika elemen yang dicari diperkirakan berada dekat dengan root. BFS juga menghasilkan urutan traversal yang mudah diamati karena berjalan level demi level.



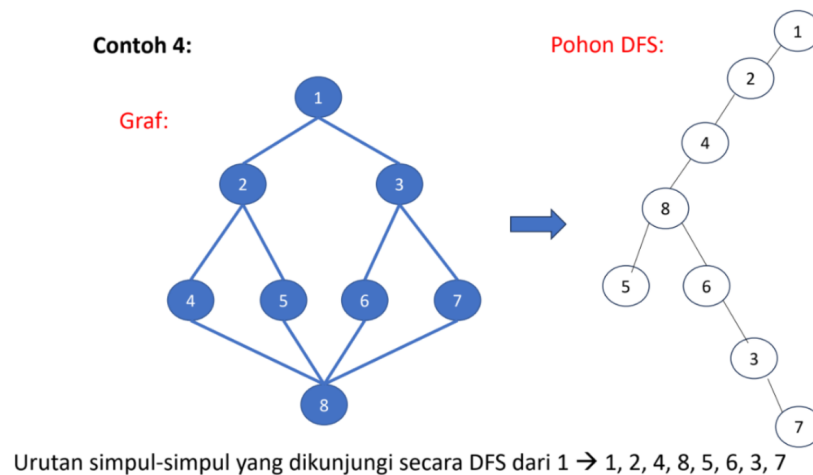
Gambar 1: Visualisasi algoritma Breadth First Search

2.2 Depth First Search

Depth First Search (DFS) adalah algoritma traversal graf atau pohon yang menelusuri satu cabang sedalam mungkin sebelum berpindah ke cabang berikutnya. Pada struktur pohon, DFS dapat dipahami sebagai penelusuran dari root menuju anak, cucu, dan seterusnya hingga mencapai leaf, lalu kembali ke cabang lain.

DFS dapat diimplementasikan secara rekursif atau iteratif. Pada aplikasi ini, DFS diimplementasikan secara iteratif menggunakan stack agar lebih aman untuk DOM Tree yang dalam. Stack mengikuti prinsip LIFO (*Last In, First Out*). Node terakhir yang dimasukkan akan diproses terlebih dahulu.

Pada DOM Tree, DFS cocok digunakan ketika elemen yang dicari diperkirakan berada pada cabang yang lebih dalam atau ketika penelusuran pre-order dibutuhkan.



Gambar 2: Visualisasi algoritma Depth First Search

2.3 Document Object Model

Document Object Model (DOM) adalah representasi terstruktur dari dokumen HTML dalam bentuk pohon. Setiap elemen HTML direpresentasikan sebagai node. Hubungan antarelemen seperti parent, child, dan sibling direpresentasikan melalui hubungan antar node pada tree.

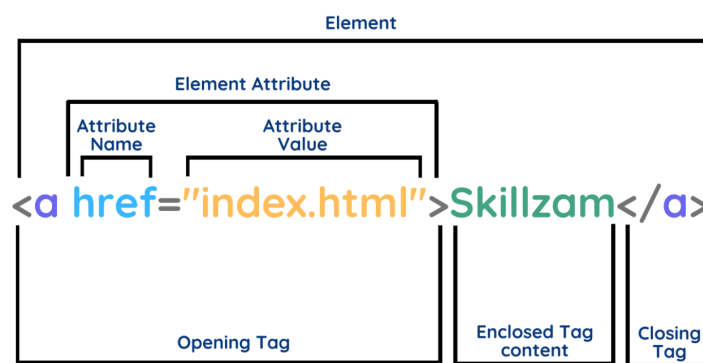
Secara umum, dokumen HTML memiliki elemen root `<html>` yang berisi `<head>` dan `<body>`. Bagian `<head>` berisi metadata, sedangkan `<body>` berisi konten utama yang ditampilkan kepada pengguna.

Dalam tugas ini, DOM Tree digunakan sebagai struktur data utama untuk proses traversal. Algoritma BFS dan DFS menelusuri node-node pada DOM Tree untuk mencari elemen yang sesuai dengan CSS selector.

2.4 Tag HTML

Tag HTML adalah penanda yang digunakan untuk mendefinisikan struktur dan jenis elemen pada dokumen HTML. Sebagian besar tag memiliki pasangan tag pembuka dan tag penutup, misalnya `<p>` dan `</p>`. Beberapa tag dapat bersifat *self-closing*.

Dalam DOM Tree, setiap tag HTML direpresentasikan sebagai node. Tag yang berada di dalam tag lain menjadi child node, sedangkan tag pembungkusnya menjadi parent node. Struktur ini membentuk relasi hierarkis yang dapat ditelusuri menggunakan algoritma BFS dan DFS.



Gambar 3: Struktur sintaks HTML

2.5 CSS Selector

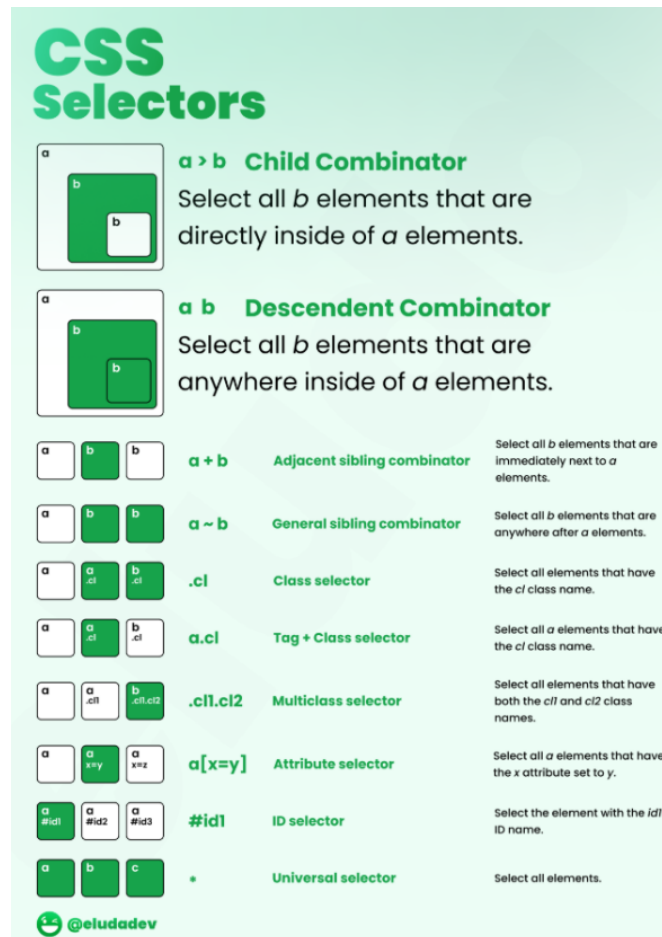
CSS selector adalah mekanisme untuk memilih elemen HTML tertentu. Dalam konteks tugas ini, selector digunakan sebagai query untuk menentukan elemen mana saja pada DOM Tree yang cocok dengan kriteria pengguna.

Beberapa selector yang digunakan dalam aplikasi ini adalah sebagai berikut:

- **Tag selector**, misalnya `p`, untuk memilih semua elemen `<p>`.
- **Class selector**, misalnya `.box`, untuk memilih elemen dengan class tertentu.
- **ID selector**, misalnya `#header`, untuk memilih elemen dengan atribut id tertentu.
- **Universal selector**, yaitu `*`, untuk memilih seluruh elemen.
- **Attribute selector**, misalnya `[href]` atau `[type=text]`.

Selector juga dapat dikombinasikan dengan combinator:

- Child combinator ($>$)
- Descendant combinator (spasi)
- Adjacent sibling combinator ($+$)
- General sibling combinator ($\sim$)



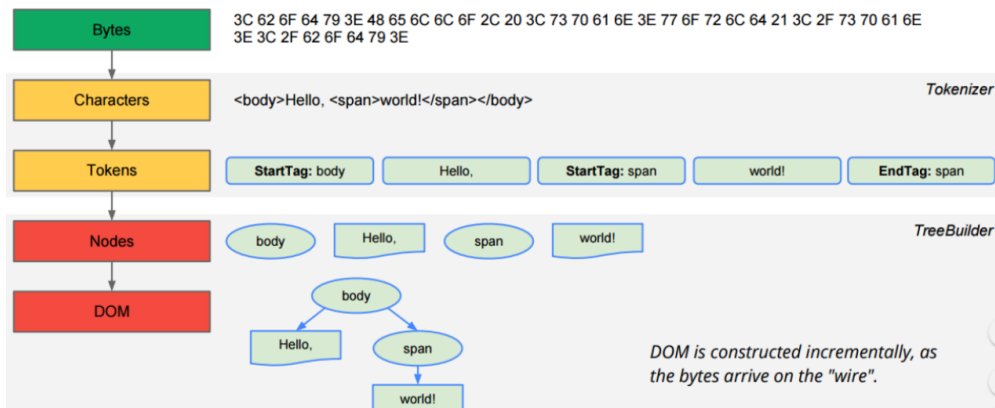
Gambar 4: Visualisasi CSS selector

2.6 HTML Parsing

HTML parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon. Parser mengenali tag pembuka, tag penutup, atribut, dan teks, lalu menyusunnya menjadi DOM Tree.

Hasil parsing digunakan oleh algoritma BFS dan DFS. Setiap node menyimpan informasi seperti id node, tag, atribut, kedalaman, dan daftar children. Informasi ini memungkinkan aplikasi menampilkan visualisasi tree dan melakukan pencocokan selector secara terstruktur.

The HTML(5) parser at work...



Gambar 5: Proses parsing HTML menjadi DOM Tree

2.7 Lowest Common Ancestor dan Binary Lifting

Lowest Common Ancestor (LCA) adalah node leluhur terendah yang sama dari dua node pada pohon. Dalam DOM Tree, LCA dapat digunakan untuk mencari elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu.

Teknik Binary Lifting mempercepat pencarian LCA dengan membuat tabel ancestor. Untuk setiap node, disimpan ancestor ke- 2^k pada tabel **up**. Setelah preprocessing, query LCA dapat dilakukan dengan menaikkan node menggunakan loncatan pangkat dua.

Preprocessing Binary Lifting memiliki kompleksitas $O(N \log N)$, dengan N sebagai jumlah node. Query LCA memiliki kompleksitas $O(\log N)$. Pada implementasi aplikasi, response LCA juga mengembalikan path untuk keperluan visualisasi, sehingga terdapat tambahan waktu sebesar panjang path yang dikembalikan.

BAB 3

APLIKASI BFS DAN DFS

3.1 Aplikasi BFS

Breadth First Search (BFS) adalah algoritma penelusuran pohon yang bekerja secara melebar. Pada DOM Tree, BFS mengunjungi seluruh node pada satu level kedalaman sebelum bergerak ke level berikutnya. Dengan demikian, node yang berada lebih dekat dengan root akan selalu diproses lebih dahulu dibandingkan node yang berada lebih dalam.

3.1.1 Konsep BFS pada DOM Tree

Saat diterapkan pada DOM Tree, BFS memandang struktur HTML sebagai pohon berakar dengan `<html>` sebagai root. Setiap elemen HTML diperlakukan sebagai node, dan hubungan antara elemen induk dengan elemen anaknya merepresentasikan tepi pohon. BFS menelusuri pohon ini lapisan demi lapisan, menjamin bahwa setiap node pada kedalaman ke- d dikunjungi sebelum node mana pun pada kedalaman ke- $(d + 1)$.

Sifat ini membuat BFS sangat sesuai untuk skenario pencarian di mana elemen yang dicari kemungkinan besar berada dekat dengan puncak dokumen HTML, seperti elemen struktural `<header>`, `<nav>`, atau elemen utama pada lapisan pertama `<body>`. BFS juga menghasilkan urutan traversal yang mudah diprediksi dan konsisten dengan hierarki dokumen.

3.1.2 Struktur Data Queue

BFS membutuhkan struktur data queue yang bekerja berdasarkan prinsip First In First Out (FIFO). Node yang lebih dulu dimasukkan ke dalam queue akan diproses lebih dulu. Prinsip inilah yang membuat BFS selalu memproses node berdasarkan urutan kedatangan, identik dengan urutan berdasarkan level kedalaman.

Queue pada implementasi ini dibangun menggunakan *linked list* dengan dua pointer, yaitu `head` yang menunjuk ke elemen paling depan dan `tail` yang menunjuk ke elemen paling belakang. Operasi penambahan node ke belakang queue (*enqueue*) dan pengambilan node dari depan queue (*dequeue*) keduanya berjalan dalam waktu $O(1)$. Perancangan ini dilakukan untuk menghindari keterbatasan operasi `Array.shift()` pada JavaScript yang

beroperasi dalam $O(n)$ akibat proses reindexing seluruh elemen array setiap kali elemen paling depan diambil. Pada DOM Tree berukuran besar dengan ribuan node, perbedaan kompleksitas ini berdampak nyata terhadap performa keseluruhan penelusuran.

3.1.3 Alur Kerja Traversal BFS

BFS dimulai dengan memasukkan root DOM Tree ke dalam queue. Kemudian, algoritma menjalankan iterasi selama queue belum kosong. Pada setiap iterasi, node paling depan diambil dari queue dan dievaluasi apakah cocok dengan selector yang diberikan. Jika cocok, node tersebut dicatat sebagai hasil pencarian. Setelah evaluasi, seluruh anak dari node yang diproses dimasukkan ke dalam queue dari kiri ke kanan sesuai urutan natural HTML.

Pola ini menjamin bahwa seluruh node pada level yang sama diproses sebelum BFS bergerak ke level berikutnya. BFS berhenti ketika queue sudah kosong atau ketika jumlah hasil yang ditemukan telah mencapai batas yang ditentukan pengguna. Apabila traversal berhenti lebih awal karena batas hasil terpenuhi, node yang masih tersisa di dalam queue dicatat sebagai **skip** pada traversal log.

3.1.4 Contoh Urutan Traversal BFS

Diberikan DOM Tree berikut:

```
html
  head
    title
  body
    h1
    p
```

BFS akan mengunjungi node dalam urutan berikut:

$$\text{html} \rightarrow \text{head} \rightarrow \text{body} \rightarrow \text{title} \rightarrow \text{h1} \rightarrow \text{p}$$

Urutan ini menunjukkan bahwa BFS memproses **head** dan **body** sebelum **title**, **h1**, dan **p** karena keduanya berada pada level yang lebih dangkal. Node-node pada level kedua baru diproses setelah seluruh node pada level pertama selesai dikunjungi.

3.1.5 Kecocokan BFS untuk Berbagai Skenario

BFS paling efektif digunakan ketika elemen yang dicari kemungkinan besar berada di bagian awal atau atas dokumen HTML. Pada pencarian dengan limit hasil yang kecil, BFS berpotensi menemukan hasil lebih cepat jika elemen target berada pada kedalaman rendah. Namun, jika elemen target berada sangat dalam pada satu cabang tertentu, BFS harus menelusuri seluruh node pada setiap level sebelum mencapai target, sehingga membutuhkan lebih banyak memori untuk menyimpan node satu level yang bisa sangat lebar.

3.2 Aplikasi DFS

Depth First Search (DFS) adalah algoritma penelusuran pohon yang bekerja secara mendalam. Pada DOM Tree, DFS menelusuri satu cabang pohon sampai ke node paling dalam sebelum kembali dan menelusuri cabang lainnya. Berbeda dengan BFS yang menjelajahi pohon secara horizontal level demi level, DFS menjelajahi pohon secara vertikal cabang demi cabang.

3.2.1 Konsep DFS pada DOM Tree

Saat diterapkan pada DOM Tree, DFS mengikuti satu jalur dari root ke leaf sebelum mempertimbangkan jalur lain. Algoritma memulai dari `<html>` sebagai root, lalu segera masuk ke anak pertama, kemudian ke anak dari anak tersebut, dan seterusnya hingga mencapai node terdalam pada cabang tersebut. Setelah mencapai node terdalam, DFS melakukan *backtracking* dan berpindah ke cabang berikutnya.

Sifat ini membuat DFS cocok untuk skenario pencarian di mana elemen yang dicari kemungkinan besar berada pada bagian dalam atau bersarang jauh dari root dokumen HTML, seperti elemen yang berada di dalam beberapa lapisan `<div>` atau elemen di dalam tabel bersarang.

3.2.2 Struktur Data Stack

DFS membutuhkan struktur data stack yang bekerja berdasarkan prinsip Last In First Out (LIFO). Node yang paling akhir dimasukkan ke dalam stack akan diproses paling pertama. Prinsip inilah yang membuat DFS selalu bergerak ke cabang terdalam sebelum

berpindah ke cabang lain.

Stack pada implementasi ini dibangun menggunakan *linked list* dengan pointer `top` yang menunjuk ke elemen paling atas. Operasi penambahan node ke atas stack (*push*) dan pengambilan node dari atas stack (*pop*) keduanya berjalan dalam waktu $O(1)$. Implementasi menggunakan linked list dipilih untuk menjaga konsistensi arsitektur dengan queue yang digunakan oleh BFS, sehingga kedua struktur data memiliki pola implementasi yang seragam di dalam sistem.

3.2.3 Reverse Push untuk Mempertahankan Urutan Natural HTML

Karena stack bersifat LIFO, jika anak-anak suatu node dimasukkan ke stack dari kiri ke kanan, maka anak paling kanan akan keluar lebih dahulu saat diproses. Akibatnya, urutan kunjungan DFS akan menjadi terbalik dari urutan natural HTML.

Untuk mengatasi hal ini, anak-anak dari setiap node dimasukkan ke dalam stack dari kanan ke kiri. Dengan cara ini, anak paling kiri akan berada di paling atas stack dan akan diproses terlebih dahulu saat diambil. Strategi ini menjamin bahwa urutan kunjungan DFS tetap mengikuti urutan alami HTML dari kiri ke kanan.

3.2.4 Pre-order Iteratif

DFS pada implementasi ini menggunakan strategi *pre-order*, yaitu node dievaluasi terhadap selector sebelum anak-anaknya dimasukkan ke dalam frontier. Pola ini membuat node induk selalu dievaluasi sebelum node anak, konsisten dengan urutan pembacaan dokumen HTML.

DFS juga diimplementasikan secara *iteratif* menggunakan stack manual, bukan secara rekursif. Pendekatan iteratif dipilih karena rekursi pada JavaScript bergantung pada *call stack* bawaan yang memiliki batas kedalaman. DOM Tree dari halaman web nyata seperti dokumentasi teknis atau Wikipedia dapat memiliki kedalaman yang sangat besar, dan rekursi pada kondisi tersebut berisiko menyebabkan *stack overflow*. Dengan menggunakan stack manual, kendali sepenuhnya berada pada program sehingga DFS dapat berjalan aman pada DOM Tree berukuran dan kedalaman berapapun.

3.2.5 Alur Kerja Traversal DFS

DFS dimulai dengan memasukkan root ke dalam stack. Selama stack belum kosong, algoritma mengambil node paling atas dari stack dan mengevaluasinya terhadap selector. Jika node cocok, node tersebut dicatat sebagai hasil pencarian. Setelah evaluasi, anak-anak dari node yang diproses dimasukkan ke stack dari kanan ke kiri. DFS berhenti ketika stack kosong atau ketika jumlah hasil telah mencapai batas yang ditentukan. Node yang masih tersisa di dalam stack saat traversal berhenti dicatat sebagai **skip** pada traversal log.

3.2.6 Contoh Urutan Traversal DFS

Dengan DOM Tree yang sama, DFS akan mengunjungi node dalam urutan berikut:

$$\text{html} \rightarrow \text{head} \rightarrow \text{title} \rightarrow \text{body} \rightarrow \text{h1} \rightarrow \text{p}$$

Urutan ini menunjukkan bahwa DFS masuk ke cabang **head** terlebih dahulu, mengunjungi **title** hingga ke node terdalam, sebelum kembali ke **html** dan masuk ke cabang **body**. Hal ini berbeda dengan BFS yang mengunjungi **head** dan **body** secara berurutan sebelum turun ke anak-anaknya.

3.3 Perbandingan BFS dengan DFS

BFS dan DFS adalah dua algoritma traversal yang memiliki filosofi penelusuran berbeda. Keduanya memulai dari titik yang sama dan menghasilkan himpunan hasil yang identik jika tidak ada batasan jumlah hasil, tetapi cara keduanya mencapai hasil tersebut sangat berbeda. Bagian ini menganalisis perbedaan keduanya secara sistematis dari berbagai aspek.

3.3.1 Cara Kerja Traversal

BFS menelusuri pohon secara horizontal, yakni memproses seluruh node pada satu level sebelum bergerak ke level berikutnya. Prinsip ini dikenal sebagai penelusuran berbasis level. Sebaliknya, DFS menelusuri pohon secara vertikal, yakni masuk ke satu cabang sedalam mungkin sebelum berpindah ke cabang lain. Prinsip ini dikenal sebagai penelusuran berbasis kedalaman.

Perbedaan cara kerja ini menghasilkan urutan kunjungan yang berbeda. BFS menghasilkan urutan kunjungan yang mencerminkan struktur level dokumen HTML, sementara DFS menghasilkan urutan kunjungan yang mencerminkan struktur cabang atau subtree dokumen HTML.

3.3.2 Kompleksitas Waktu

Baik BFS maupun DFS memiliki kompleksitas waktu $O(N)$ dalam kasus terburuk, di mana N adalah jumlah seluruh node pada DOM Tree. Ini karena dalam kondisi tanpa limit hasil, kedua algoritma harus mengunjungi setiap node tepat satu kali untuk memastikan semua elemen yang cocok dengan selector ditemukan.

Namun, ketika pencarian dibatasi dengan limit hasil, perbedaan waktu antara BFS dan DFS menjadi bergantung pada posisi elemen target di dalam DOM Tree. Jika elemen yang dicari berada pada level dangkal, BFS cenderung menemukannya lebih cepat karena memproses seluruh node dangkal terlebih dahulu. Sebaliknya, jika elemen yang dicari berada jauh di dalam satu cabang, DFS cenderung lebih cepat karena langsung menyelami cabang tersebut tanpa harus memproses seluruh node pada setiap level.

3.3.3 Kompleksitas Memori

BFS harus menyimpan seluruh node pada satu level di dalam queue sebelum dapat memproses level berikutnya. Pada DOM Tree yang sangat lebar seperti halaman yang memiliki satu elemen `<body>` dengan ratusan anak langsung, ukuran queue pada satu waktu dapat tumbuh sangat besar. Dalam kasus ekstrem, BFS membutuhkan memori $O(W)$ di mana W adalah lebar maksimum pohon.

DFS sebaliknya hanya perlu menyimpan node-node sepanjang satu jalur dari root ke node yang sedang diproses beserta sibling-sibling yang belum dikunjungi. Pada DOM Tree yang sangat dalam, DFS membutuhkan memori $O(D)$ di mana D adalah kedalaman maksimum pohon. Pada DOM Tree yang seimbang, D umumnya jauh lebih kecil dibandingkan W sehingga DFS lebih hemat memori pada pohon yang lebar.

3.3.4 Pengaruh Karakteristik DOM Tree

Pada DOM Tree yang lebar, yaitu pohon dengan banyak anak pada setiap node tetapi kedalaman yang tidak terlalu besar seperti halaman beranda sebuah situs berita, BFS

membutuhkan lebih banyak memori karena harus menyimpan semua node anak pada setiap level. DFS lebih hemat memori dalam kondisi ini karena hanya perlu menyimpan satu jalur aktif pada satu waktu.

Pada DOM Tree yang dalam, yaitu pohon dengan kedalaman yang besar tetapi tidak terlalu banyak cabang pada setiap node seperti halaman dokumentasi dengan banyak elemen bersarang, DFS membutuhkan lebih banyak memori stack untuk menyimpan jalur aktif yang panjang. BFS lebih efisien dari sisi memori dalam kondisi ini karena lebar pohon yang kecil membuat ukuran queue tetap terjaga.

3.3.5 Pengaruh Limit Hasil Pencarian

Limit hasil pencarian berdampak berbeda terhadap efisiensi BFS dan DFS tergantung pada distribusi elemen yang cocok di dalam DOM Tree.

Jika elemen-elemen yang cocok dengan selector tersebar merata pada level dangkal dokumen, BFS menemukan hasil limit lebih cepat karena langsung mengunjungi semua node dangkal. Sebaliknya, DFS mungkin harus menyelami cabang-cabang yang tidak mengandung hasil sebelum menemukan semua hasil yang dibutuhkan.

Jika elemen-elemen yang cocok berada pada bagian dalam satu atau beberapa cabang spesifik, DFS menemukan hasil lebih cepat karena langsung masuk ke cabang terdalam. BFS harus memproses seluruh node pada setiap level terlebih dahulu sebelum mencapai node yang dalam.

3.3.6 Kecocokan Algoritma untuk Berbagai Skenario Selector

Untuk pencarian menggunakan *id selector* (`#main`) atau elemen struktural (`header`, `nav`), elemen ini biasanya berada pada level yang tidak terlalu dalam dari root. BFS lebih cocok digunakan karena menemukan elemen tersebut dengan lebih sedikit iterasi.

Untuk pencarian menggunakan *class selector* atau *attribute selector* yang elemen targetnya tersebar di berbagai kedalaman dokumen, DFS dan BFS menghasilkan performa yang sebanding dalam kasus terburuk. Namun, DFS berpotensi lebih efisien jika elemen target terkonsentrasi pada cabang-cabang tertentu yang dalam.

Untuk pencarian dengan *descendant combinator* (`div p`) pada dokumen dengan banyak elemen bersarang, DFS menelusuri setiap subtree secara tuntas sebelum berpindah ke subtree lain. Ini membuat DFS lebih mudah diprediksi urutannya karena seluruh hasil

pada satu subtree ditemukan sebelum pindah ke subtree berikutnya.

3.3.7 Tabel Perbandingan

Aspek	BFS	DFS
Arah penelusuran	Horizontal, level demi level	Vertikal, cabang demi cabang
Struktur data	Queue (FIFO)	Stack (LIFO)
Implementasi	Iteratif dengan queue	Iteratif dengan stack manual
Kompleksitas waktu	$O(N)$	$O(N)$
Memori pada pohon lebar	Besar, menyimpan satu level	Lebih kecil
Memori pada pohon dalam	Lebih kecil	Besar, menyimpan satu jalur
Pencarian elemen dangkal	Lebih cepat	Lebih lambat
Pencarian elemen dalam	Lebih lambat	Lebih cepat
Urutan hasil	Berdasarkan level	Berdasarkan cabang

Secara umum, tidak ada algoritma yang selalu lebih baik dari yang lain. Pemilihan antara BFS dan DFS sebaiknya disesuaikan dengan karakteristik DOM Tree dan posisi perkiraan elemen yang ingin dicari. Itulah mengapa aplikasi ini menyediakan kedua algoritma sekaligus dan memberikan pilihan kepada pengguna.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Program Secara Umum

Program yang dibuat adalah aplikasi web untuk menelusuri DOM Tree menggunakan CSS selector. Aplikasi menerima dua jenis masukan, yaitu URL website dan teks HTML manual. Setelah input diterima, backend mengambil atau membaca HTML, melakukan parsing menjadi DOM Tree, lalu menjalankan pencarian elemen menggunakan algoritma BFS atau DFS sesuai pilihan pengguna.

Project dibagi menjadi dua bagian utama, yaitu backend dan frontend. Backend menggunakan TypeScript dengan Express. Frontend menggunakan React dengan Vite. Selain itu, project juga sudah memiliki konfigurasi Docker melalui `Dockerfile` pada backend dan frontend serta `docker-compose.yml` pada root project.

4.2 Struktur Direktori dan Modul Program

Struktur project disusun secara modular. Backend menangani scraping, parsing, traversal, selector matching, dan API. Frontend menangani input pengguna, pemanggilan API, visualisasi tree, animasi traversal, dan tampilan traversal log.

File atau Modul	Fungsi
<code>backend/src/index.ts</code>	Mengatur server Express dan endpoint API <code>/api/scrape</code> serta <code>/api/search</code> .
<code>backend/src/scrapper/scrapper.ts</code>	Mengambil HTML dari URL menggunakan request HTTP dan header yang menyerupai browser.
<code>backend/src/parser/parser.ts</code>	Mengubah HTML menjadi DOM Tree menggunakan tokenisasi dari <code>parse5</code> , lalu membangun struktur tree sendiri.
<code>backend/src/algorithm/bfs.ts</code>	Menjalankan pencarian menggunakan algoritma BFS.

<code>backend/src/algorithm/dfs.ts</code>	Menjalankan pencarian menggunakan algoritma DFS.
<code>backend/src/algorithm/searchCore.ts</code>	Menyediakan loop traversal bersama untuk BFS dan DFS.
<code>backend/src/algorithm/selector.ts</code>	Melakukan tokenisasi, parsing, dan pencocokan CSS selector.
<code>backend/src/utils/queue.ts</code>	Menyediakan struktur data queue berbasis linked list.
<code>backend/src/utils/stack.ts</code>	Menyediakan struktur data stack berbasis linked list.
<code>backend/src/utils/logger.ts</code>	Mencatat setiap langkah traversal ke dalam traversal log.
<code>backend/src/utils/utils.ts</code>	Menyediakan fungsi bantu seperti perhitungan kedalaman maksimum tree.
<code>backend/src/types/dom.ts</code>	Mendefinisikan tipe data DOM node, traversal log, dan respons pencarian.
<code>frontend/src/components/InputParser.ts</code>	Melampirkan form input URL atau HTML, pilihan algoritma, CSS selector, dan limit hasil.
<code>frontend/src/components/OutputParser.ts</code>	Memanggil API backend, menyimpan hasil scrape dan pencarian, serta mengatur animasi traversal.
<code>frontend/src/components/DOMTreeViewer.ts</code>	Menampilkan visualisasi DOM Tree menggunakan <code>react-d3-tree</code> .
<code>frontend/src/components/LogRow.ts</code>	Menampilkan satu baris traversal log.
<code>frontend/src/utils/convertDomTree.ts</code>	Mengubah struktur DOM backend menjadi format tree yang dapat dibaca oleh <code>react-d3-tree</code> .
<code>docker-compose.yml</code>	Menjalankan backend dan frontend secara bersamaan menggunakan Docker Compose.

4.3 Implementasi Backend

Backend dibuat dengan TypeScript dan Express. File `index.ts` mendefinisikan dua endpoint utama. Endpoint `/api/scrape` menerima input berupa URL atau teks HTML. Jika input berupa URL, backend memanggil fungsi `scrapeURL`. Jika input berupa HTML manual, backend langsung memproses string HTML tersebut. Hasilnya kemudian diparsing menjadi DOM Tree dan nilai kedalaman maksimum tree.

Endpoint `/api/search` menerima DOM Tree, pilihan algoritma, selector, dan limit. Backend memvalidasi request terlebih dahulu. Jika algoritma bernilai `bfs`, backend memanggil `bfsSearch`. Jika algoritma bernilai `dfs`, backend memanggil `dfsSearch`. Respons endpoint berisi `results`, `traversalLog`, dan `stats`.

4.4 Implementasi Scraper dan Parser

Modul scraper bertugas mengambil HTML dari URL. URL divalidasi terlebih dahulu agar hanya protokol `http` dan `https` yang diterima. Request menggunakan header `User-Agent`, `Accept`, dan `Accept-Language` agar lebih sesuai dengan request browser. Scraper juga menerapkan batas waktu request untuk menghindari proses yang terlalu lama.

Modul parser menggunakan `parse5` untuk membaca dokumen HTML. Library ini digunakan sebagai tokenizer dan parser HTML dasar, sedangkan struktur DOM Tree aplikasi tetap dibangun sendiri melalui fungsi konversi. Node teks dan komentar diabaikan. Setiap node elemen menyimpan id unik, tag, atribut, children, depth, dan `innerText` jika ada.

4.5 Implementasi BFS dan DFS

BFS dan DFS menggunakan struktur yang sama melalui `searchCore.ts`. Perbedaannya hanya terdapat pada frontier. BFS memakai queue, sedangkan DFS memakai stack. Pada BFS, anak-anak node dimasukkan sesuai urutan normal. Pada DFS, anak-anak node dimasukkan dengan urutan terbalik agar anak paling kiri tetap diproses lebih dahulu.

```
const frontier: Frontier<DOMNode> = {
  add: (node) => queue.enqueue(node),
  remove: () => queue.dequeue(),
```

```
isEmpty: () => queue.isEmpty(),  
};
```

Fungsi `runTraversal` mencatat waktu eksekusi, jumlah node yang benar-benar dikunjungi, hasil pencarian, dan traversal log. Jika jumlah hasil sudah mencapai limit, traversal berhenti lebih awal. Node yang masih tersisa pada frontier dicatat sebagai `skip`.

4.6 Implementasi CSS Selector

Selector diproses oleh `selector.ts`. Implementasi selector dibuat sendiri melalui tokenizer dan parser. Selector kemudian dicocokkan terhadap node DOM dengan strategi right-to-left. Dengan strategi ini, node kandidat tetap ditemukan oleh BFS atau DFS, sedangkan matcher hanya mengecek apakah node tersebut memenuhi konteks selector.

Selector yang didukung meliputi tag selector, class selector, ID selector, universal selector, attribute selector, compound selector, grouping, child combinator, descendant combinator, adjacent sibling combinator, general sibling combinator, serta pseudo-class `:first-child` dan `:last-child`.

4.7 Implementasi Frontend

Frontend menggunakan React dan Vite. Komponen `InputPanel.jsx` menyediakan form untuk memilih jenis input, memasukkan URL atau HTML, memilih algoritma, memasukkan CSS selector, dan menentukan limit hasil. Setelah form valid, data dikirim ke halaman visualisasi.

Komponen `OutputPanel.jsx` memanggil endpoint `/api/scrape` terlebih dahulu untuk memperoleh DOM Tree. Setelah DOM Tree tersedia, frontend memanggil endpoint `/api/search` dengan algoritma, selector, dan limit yang dipilih pengguna. Hasil pencarian kemudian disimpan dan digunakan untuk menampilkan visualisasi tree, traversal log, statistik pencarian, serta animasi traversal.

4.8 Implementasi Visualisasi dan Traversal Log

Visualisasi DOM Tree menggunakan `react-d3-tree`. Data dari backend diubah oleh `convertDomToTree.js` agar sesuai dengan format yang dibutuhkan oleh library tersebut.

Node yang cocok dengan selector ditandai sebagai **match**, sedangkan node yang hanya dikunjungi ditandai sebagai **visited**.

Traversal log ditampilkan menggunakan komponen `LogRow.jsx`. Setiap log berisi nomor langkah, aksi, tag, id node, dan depth. Aksi **visit** menunjukkan node dikunjungi tetapi tidak cocok, aksi **match** menunjukkan node cocok dengan selector, dan aksi **skip** menunjukkan node tidak sempat diproses karena limit sudah tercapai.

4.9 Implementasi Docker dan Deployment

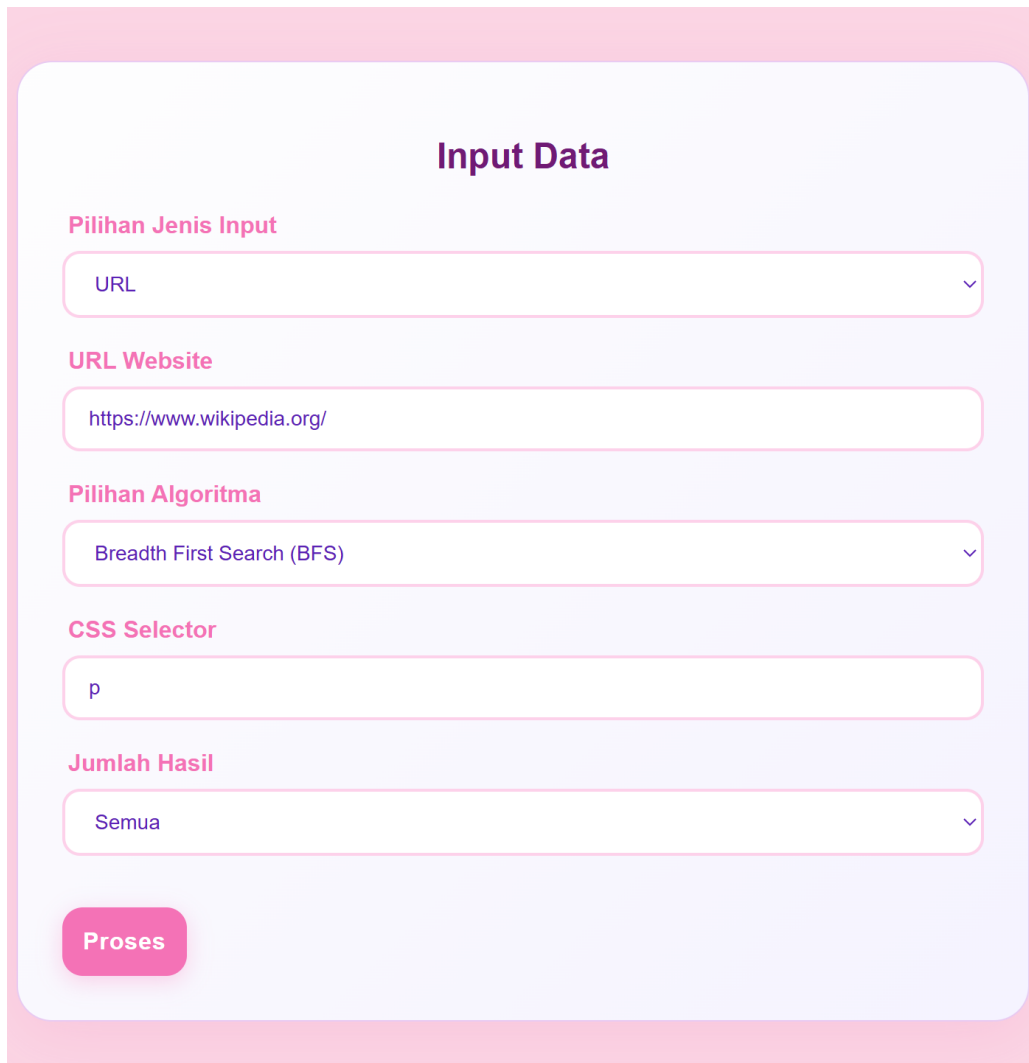
Project menyediakan `Dockerfile` untuk backend dan frontend. Backend dijalankan pada port 3000, sedangkan frontend dibangun sebagai aplikasi statis dan disajikan melalui Nginx pada port 80 di dalam container. File `docker-compose.yml` menghubungkan kedua service sehingga aplikasi dapat dijalankan dengan satu perintah.

Pada pengujian lokal, backend dapat dijalankan dengan `npm run dev`. Frontend dapat dijalankan dengan `npm run dev` dari folder frontend. Untuk Docker, aplikasi dijalankan menggunakan perintah `docker compose up -build` dari root project.

4.10 Pengujian Program

Pengujian dilakukan untuk memastikan aplikasi dapat menerima input, menjalankan algoritma pencarian, menampilkan hasil traversal, dan menampilkan visualisasi DOM Tree dengan benar. Setiap pengujian terdiri atas gambar input, gambar output, dan keterangan hasil pengujian.

1. Pengujian 1



The image shows a web form titled "Input Data" with a light purple background and rounded corners. It contains five input fields, each with a label above it: "Pilihan Jenis Input" (dropdown menu with "URL" selected), "URL Website" (text input with "https://www.wikipedia.org/"), "Pilihan Algoritma" (dropdown menu with "Breadth First Search (BFS)" selected), "CSS Selector" (text input with "p"), and "Jumlah Hasil" (dropdown menu with "Semua" selected). At the bottom left of the form is a pink button labeled "Proses".

Input Data

Pilihan Jenis Input

URL

URL Website

https://www.wikipedia.org/

Pilihan Algoritma

Breadth First Search (BFS)

CSS Selector

p

Jumlah Hasil

Semua

Proses

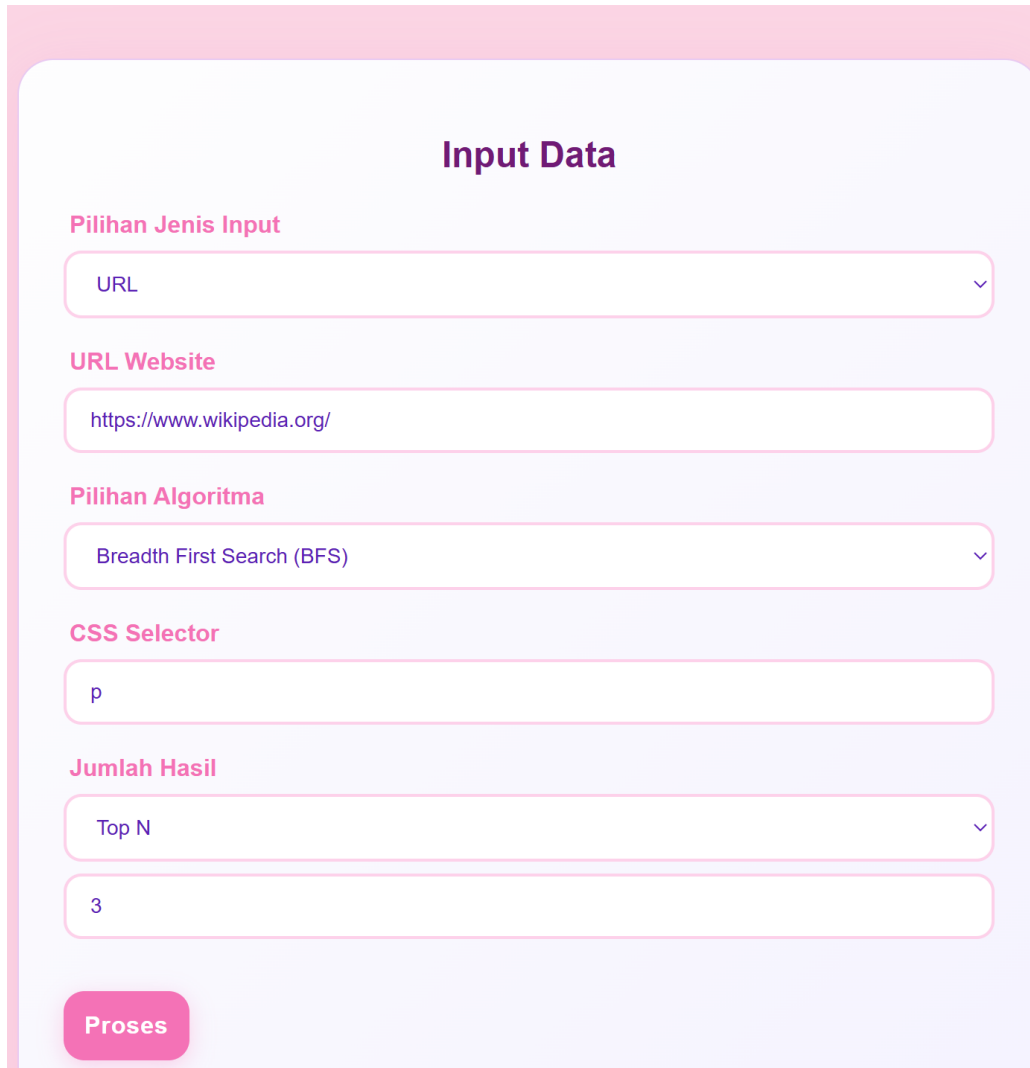
Gambar 6: Input pengujian 1



Gambar 7: Output pengujian 1

Keterangan: Saat memasukkan URL sebuah website untuk menemukan semua node dengan CSS selector `<p>` untuk semua hasil pencarian menggunakan algoritma BFS, ditampilkan informasi hasil semua pencarian, visualisasi DOM tree, hasil elemen yang cocok, dan hasil traversal.

2. Pengujian 2



The image shows a web form titled "Input Data" with a light purple background and rounded corners. It contains several input fields and a button. The fields are labeled in pink text: "Pilihan Jenis Input", "URL Website", "Pilihan Algoritma", "CSS Selector", and "Jumlah Hasil". The "Pilihan Jenis Input" field has a dropdown menu with "URL" selected. The "URL Website" field contains the text "https://www.wikipedia.org/". The "Pilihan Algoritma" field has a dropdown menu with "Breadth First Search (BFS)" selected. The "CSS Selector" field contains the text "p". The "Jumlah Hasil" field has a dropdown menu with "Top N" selected. Below the "Jumlah Hasil" field is another input field containing the number "3". At the bottom left of the form is a pink button with the text "Proses".

Input Data

Pilihan Jenis Input

URL

URL Website

https://www.wikipedia.org/

Pilihan Algoritma

Breadth First Search (BFS)

CSS Selector

p

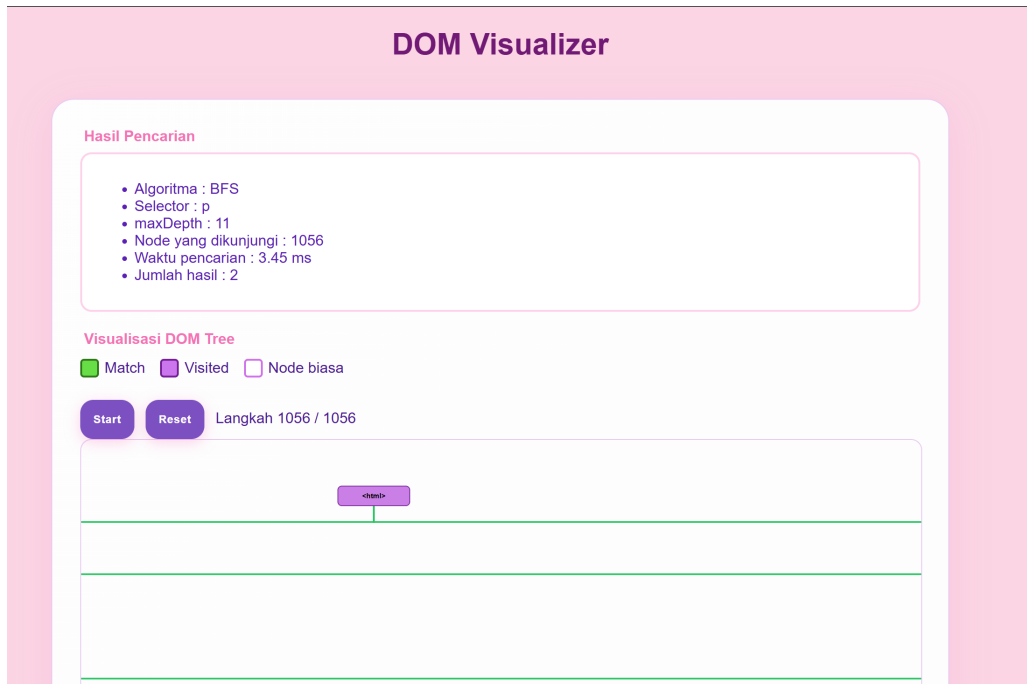
Jumlah Hasil

Top N

3

Proses

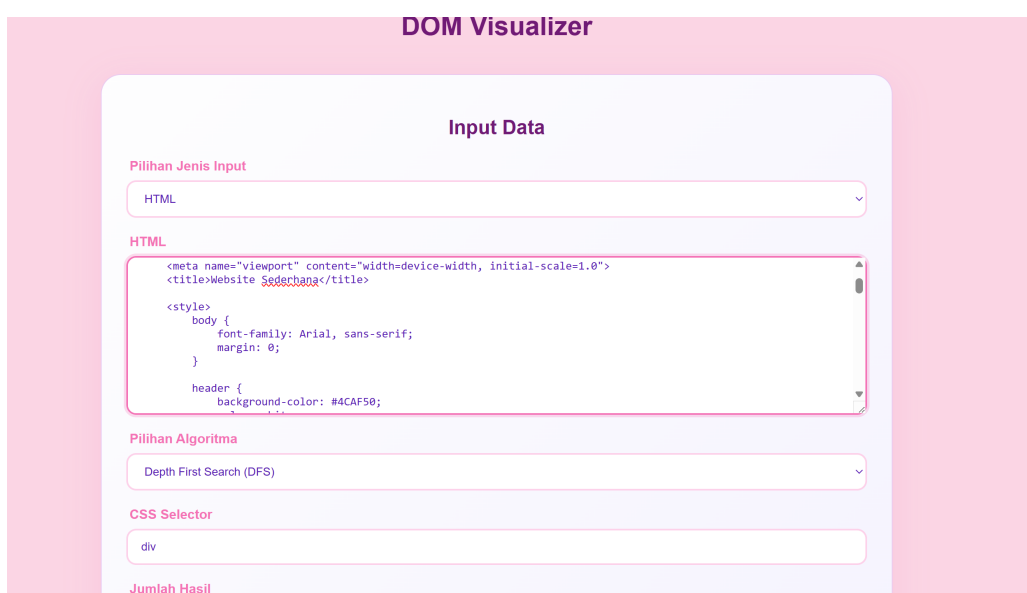
Gambar 8: Input pengujian 2



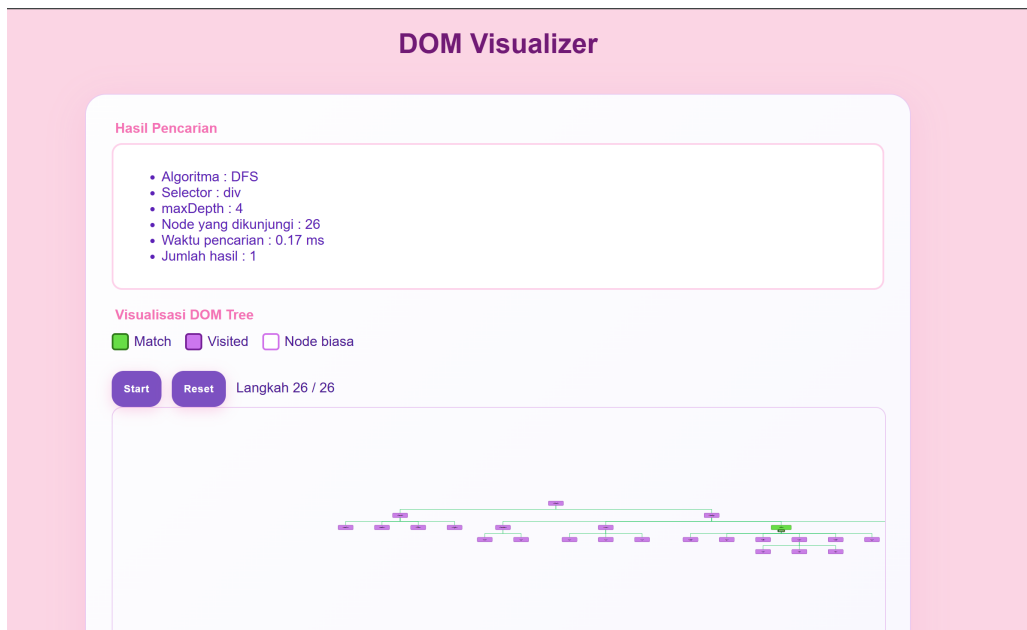
Gambar 9: Output pengujian 2

Keterangan: Saat memasukkan URL sebuah website untuk menemukan semua node dengan CSS selector <p> untuk 3 hasil pencarian teratas menggunakan algoritma BFS, ditampilkan informasi hasil pencarian, visualisasi DOM tree, 3 hasil elemen yang cocok, dan hasil traversal

3. Pengujian 3



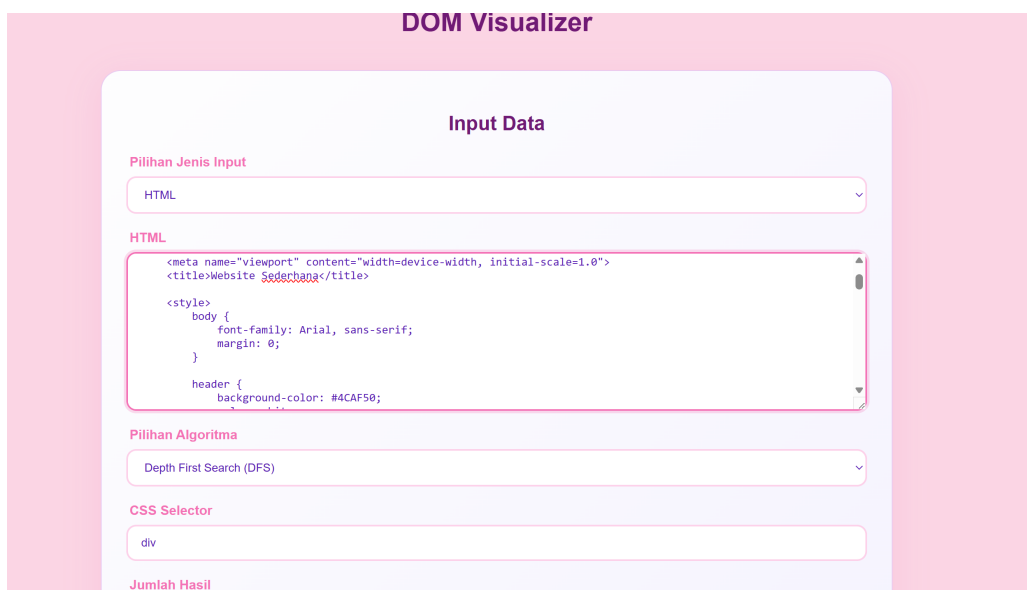
Gambar 10: Input pengujian 3



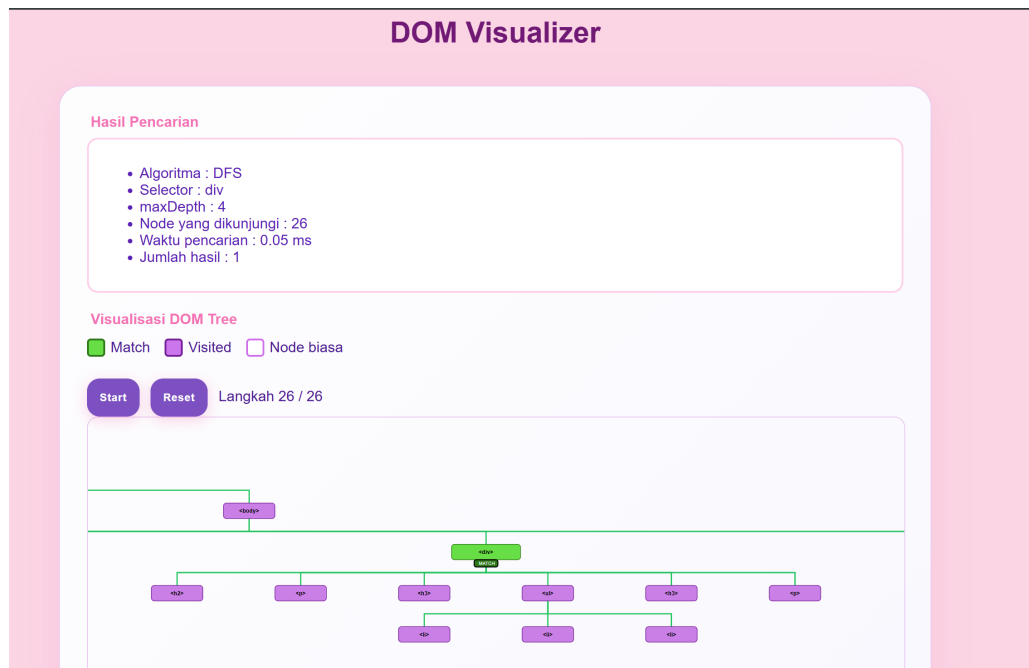
Gambar 11: Output pengujian 3

Keterangan: Saat memasukkan HTML sebuah website untuk menemukan semua node dengan CSS selector <div> untuk semua hasil pencarian menggunakan algoritma DFS, ditampilkan informasi hasil semua pencarian, visualisasi DOM tree, hasil elemen yang cocok, dan hasil traversal

4. Pengujian 4



Gambar 12: Input pengujian 4



Gambar 13: Output pengujian 4

Keterangan: Saat memasukkan HTML sebuah website untuk menemukan semua node dengan CSS selector `<div>` untuk 3 hasil pencarian teratas menggunakan algoritma DFS, ditampilkan informasi hasil pencarian, visualisasi DOM tree, 3 hasil elemen yang cocok, dan hasil traversal

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang dilakukan, algoritma BFS dan DFS dapat digunakan untuk melakukan penelusuran elemen pada DOM Tree berdasarkan CSS selector. BFS menelusuri tree secara melebar berdasarkan kedalaman, sedangkan DFS menelusuri tree secara mendalam pada satu cabang sebelum berpindah ke cabang lain.

Implementasi modular dengan `searchCore.ts` mengurangi duplikasi kode antara BFS dan DFS. CSS selector diproses melalui tokenizer, parser, dan matcher yang dibuat sendiri. Traversal log membantu frontend menampilkan proses pencarian secara lebih jelas, termasuk membedakan node `visit`, `match`, dan `skip`.

Aplikasi juga berhasil diintegrasikan sebagai aplikasi web berbasis React dan Express. Backend menangani scraping, parsing, dan pencarian, sedangkan frontend menampilkan input pengguna, visualisasi DOM Tree, traversal log, statistik pencarian, dan animasi traversal.

5.2 Saran

Pengembangan selanjutnya dapat menambahkan dukungan CSS selector yang lebih luas, optimasi visualisasi untuk DOM Tree yang sangat besar, serta peningkatan fitur bonus seperti multithreading dan LCA Binary Lifting. Selain itu, konfigurasi URL backend pada frontend dapat dibuat berbasis environment variable agar lebih mudah dipindahkan antara mode lokal dan deployment.

LAMPIRAN

Tautan Repository

https://github.com/keirzka/Tubes2_YangPentingAda.git

Tautan Video YouTube

https://youtu.be/uHDwzvOixdI?si=0_iyKDAvtSjwOEG8

Pembagian Tugas

Nama	NIM	Deskripsi Tugas
Aziza Dharma Putri	13524017	Mengerjakan laporan bagian penerapan algoritma BFS/DFS dan implementasi algoritma BFS/DFS.
Keisha Rizka Syofyani	13524073	Mengerjakan laporan bagian deskripsi tugas, landasan teori, dan pengujian. Membuat implementasi algoritma BFS dan DFS. Membuat implementasi front-end website.
Vara Azzara Ramli Pulu- kadang	13524091	Mengerjakan laporan bagian kesimpulan dan saran serta merapikan dokumen laporan. Membuat implementasi backend website.

Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Aplikasi berhasil dikompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	

4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	Bonus video	✓	
10	Bonus deploy aplikasi	✓	
11	Bonus animasi penelusuran	✓	
12	Bonus multithreading		✓
13	Bonus LCA Binary Lifting		✓

Pustaka

- [1] React Dev. *Quick Start React*. Tersedia pada <https://id.react.dev/learn>. Diakses pada 19 April 2026.
- [2] W3Schools. *JavaScript HTML DOM*. Tersedia pada <https://www.w3schools.com/jsref/>. Diakses pada 19 April 2026.
- [3] TechAltum. *HTML Tags List*. Tersedia pada <https://tutorial.techaltum.com/htmlTags.html>. Diakses pada 19 April 2026.
- [4] u/shubham_jain. *CSS Selectors Visually Explained*. Reddit r/webdev, 2022. Tersedia pada https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/. Diakses pada 19 April 2026.
- [5] Stack Overflow. *How does a parser, for example HTML, work?*. Tersedia pada <https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>. Diakses pada 19 April 2026.
- [6] R. Munir. *Breadth/Depth First Search (BFS/DFS)*. Bahan kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, STEI-ITB, 2026.
- [7] CP-Algorithms. *Lowest Common Ancestor - Binary Lifting*. Tersedia pada https://cp-algorithms.com/graph/lca_binary_lifting.html. Diakses pada 25 April 2026.

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan generatif, melainkan hasil pemikiran, diskusi, dan analisis mandiri anggota kelompok.



Aziza Dharma Putri
13524017



Vara Azzara Ramli
Pulukadang
13524091



Keisha Rizka Syofyani
13524073